

Yes — your project is doable, but you should **rewrite a few features so they obey the starter-file restriction**.

Your uploaded starter file already gives you the main allowed structure: `draw_text`, `draw_shapes`, `keyboardListener`, `specialKeyListener`, `mouseListener`, `setupCamera`, `idle`, `showScreen`, and `main`, plus OpenGL calls such as `glBegin`, `glEnd`, `glColor3f`, `glVertex3f`, `glTranslatef`, `glRotatef`, `glScalef`, `glPushMatrix`, `glPopMatrix`, `gluLookAt`, `gluPerspective`, `glutSolidCube`, `gluCylinder`, `gluSphere`, and GLUT input/display callbacks. It also already uses `GLUT_DEPTH` and clears the depth buffer, so adding depth testing is reasonable if your instructor allows “GL depth.”

Important correction to your proposal

Because you are only allowed to use functions already shown in the `.py` file, you probably **cannot directly use**:

```
GL_FOG
GL_LIGHTING
GL_LIGHT0
glEnable(GL_LIGHTING)
glLightfv(...)
glFogf(...)
glFogfv(...)
```

So instead of saying:

Depth-Based Fog using `GL_FOG`
Submarine Headlights using `GL_LIGHTING`

you should describe them as **simulated effects** using allowed functions.

A safer version would be:

Simulated Depth Fog: The underwater environment becomes darker as the submarine moves deeper by changing object colors using `glColor3f`.

Simulated Headlight Cone: A bright forward-facing visual cone/cylinder/sphere effect is drawn in front of the submarine using `gluCylinder`, `gluSphere`, `glTranslatef`, and `glRotatef`.

That keeps the idea but avoids banned OpenGL lighting/fog functions.

Revised Project Scope Under the Allowed Functions

Project Title

3D Deep Sea Rescue: Submarine Navigator

Allowed-Function-Friendly Description

This project is a 3D underwater rescue game built using PyOpenGL and GLUT. The player controls a submarine inside a deep-sea cavern system. The goal is to collect stranded rescue pods before oxygen runs out while avoiding walls, sea mines, currents, and limited visibility.

The game uses 3D transformations, primitive shapes, camera control with `gluLookAt`, keyboard/mouse input, collision math, and UI text rendering. Since advanced OpenGL lighting and fog functions are not allowed, underwater atmosphere will be simulated through object color changes, darkening effects, bubbles, and a headlight-like visual guide.

Feature Feasibility Check

Original Feature	Allowed ?	How to Implement Under Restrictions
3D Cavern Network	Yes	Use scaled <code>glutSolidCube</code> objects as walls, floor, ceiling, and blocks.
Depth-Based Fog using <code>GL_FOG</code>	Modify	Simulate by changing colors with <code>glColor3f</code> based on submarine depth.
Bubble Particles	Yes	Draw small <code>gluSphere</code> bubbles that move upward every frame.
Submarine Headlights using <code>GL_LIGHTING</code>	Modify	Draw a visible cone/cylinder/sphere in front of the submarine. No real lighting.

6-DOF Hydrodynamics	Yes	Use Python variables for position, rotation, velocity, and friction.
Dynamic Ocean Currents	Yes	Use invisible coordinate zones that add velocity to the submarine.
Dual Camera System	Yes	Use <code>gluLookAt</code> for first-person and third-person modes.
Oxygen Timer	Yes	Use a Python variable and display it with <code>draw_text</code> .
Rescue Pods	Yes	Draw pods using spheres/cylinders and check distance collision.
Bioluminescent Sponges	Yes	Draw glowing-looking colored spheres. Restore oxygen on collision.
Upgrade Crates	Yes	Draw small cubes. Apply stat upgrade when collected.
3D Bounding Collisions	Yes	Use Python AABB/sphere collision math.
Patrolling Hazards	Yes	Move hazards toward player if close. Draw with spheres/cylinders.
Destructible Terrain	Yes	Store fragile walls in a list and remove them after torpedo collision.
Sonar Pulse Radar	Modify	Use expanding <code>GL_POINTS</code> sphere or low-detail <code>gluSphere</code> ; avoid transparency/wireframe unless allowed.

Recommended Final Feature List

You can keep all 15 features, but rename the restricted ones like this:

Group 1: Environment & Atmosphere

1. **3D Cavern Network**
Built using scaled cubes and cylinders.
2. **Simulated Depth Darkness**
Instead of `GL_FOG`, use darker colors when the submarine goes deeper.

3. **Procedural Bubble Particles**
Small spheres rise upward from the cavern and submarine.
4. **Fake Submarine Headlight Beam**
A visible cone/cylinder in front of the submarine shows its direction.

Group 2: Physics & Movement

5. **6-DOF Hydrodynamic Movement**
Position and velocity on X, Y, Z with yaw and pitch.
6. **Dynamic Ocean Currents**
Invisible zones push the submarine sideways or upward/downward.
7. **Dual Camera System**
Toggle first-person and third-person camera using `gluLookAt`.

Group 3: Objectives & Survival

8. **Oxygen Timer**
Decreases over time. Reaching zero causes game over.
9. **Rescue Pods**
Collect all pods to win.
10. **Bioluminescent Sponges**
Restore oxygen when collected.
11. **Upgrade Crates**
Improve submarine movement or reduce oxygen loss.

Group 4: Dangers & Combat

12. **3D Collision System**
Detect collisions with walls, floor, ceiling, hazards, pods, and pickups.
13. **Patrolling Sea Mines / Anglerfish**
Hazards move toward the player when nearby.
14. **Destructible Rock Walls**
Torpedoes remove fragile obstacles.
15. **Sonar Pulse Radar**
Expanding pulse made from points or a sphere highlights nearby hidden objects.

Better Individual Contribution Split

Your original split is good, but I would slightly adjust it to match the restriction.

Member 1: Movement, Camera, and Atmosphere

Responsibilities:

- Submarine position, velocity, yaw, pitch
- Friction / hydrodynamic drifting
- Ocean current zones
- First-person and third-person camera with gluLookAt
- Simulated depth darkness
- Fake headlight beam

Main functions they will edit:

```
keyboardListener()  
specialKeyListener()  
mouseListener()  
setupCamera()  
idle()  
drawSubmarine()  
drawHeadlight()
```

Member 2: Collision, Hazards, Combat, Effects

Responsibilities:

- Wall and object collision
- Oxygen penalty on crash
- Moving hazards
- Torpedoes
- Destructible walls
- Sonar pulse
- Bubble particles

Main functions they will write/edit:

```
checkCollisions()  
updateHazards()  
drawHazards()  
fireTorpedo()  
updateTorpedoes()  
drawTorpedoes()
```

```
updateBubbles()
drawBubbles()
drawSonarPulse()
```

Member 3: World, Objectives, UI, Game State

Responsibilities:

- Cavern layout
- Rescue pod placement
- Sponge placement
- Upgrade crate placement
- Oxygen timer
- Score
- Win/game-over state
- UI text

Main functions they will write/edit:

```
drawCavern()
drawRescuePods()
drawPickups()
updateGameState()
drawUI()
resetGame()
```

Suggested Global Variables

You can add Python variables and lists. That should be allowed because they are not OpenGL functions.

```
# Submarine state
sub_x = 0
sub_y = 0
sub_z = 80

sub_yaw = 0
sub_pitch = 0
```

```
vel_x = 0
vel_y = 0
vel_z = 0

thrust_power = 0.8
turn_speed = 3
friction = 0.96

# Game state
oxygen = 100
score = 0
game_over = False
game_won = False

# Camera
first_person = False

# Objects
walls = []
rescue_pods = []
sponges = []
upgrade_crates = []
hazards = []
torpedoes = []
bubbles = []
currents = []
fragile_walls = []

# Effects
sonar_active = False
sonar_radius = 0
```

Controls Suggestion

Use keys that are easy to explain in your demo.

Key	Action
W	Thrust forward

S	Reverse thrust
A	Turn left
D	Turn right
Q	Move upward
E	Move downward
Arrow Up	Pitch nose down/up depending on your coordinate setup
Arrow Down	Pitch opposite direction
Left Mouse	Fire torpedo
Right Mouse	Toggle camera
F	Activate sonar pulse
R	Reset game

How to Implement the Restricted Effects

1. Simulated Depth Fog

Instead of real `GL_FOG`, calculate a darkness factor.

Example idea:

```
depth_factor = max(0.2, 1.0 - abs(sub_z) / 600)
glColor3f(0.0, 0.2 * depth_factor, 0.4 * depth_factor)
```

Objects farther/deeper can be drawn darker. This gives the visual feeling of deep water without using `GL_FOG`.

2. Fake Headlight

Use a cone-like cylinder in front of the submarine.

Since the file already uses:

```
gluCylinder(...)
glTranslatef(...)
glRotatef(...)
```

you can draw a narrow beam in front of the submarine. It will not actually illuminate walls, but visually it communicates the headlight direction.

3. Sonar Pulse

Avoid `glutWireSphere` or transparency unless allowed.

Use `GL_POINTS`, since your starter file already uses it. You can draw expanding points around the submarine.

Concept:

```
glPointSize(3)
glBegin(GL_POINTS)
# generate points around a sphere using math.sin and math.cos
glEnd()
```

This gives the feeling of an expanding sonar shell.

4. Ocean Currents

Use invisible zones.

Example data structure:

```
currents = [
  {"x1": -300, "x2": -100, "y1": 100, "y2": 300, "z1": 0, "z2": 200, "push": (0.2, 0.0, 0.1)}
]
```

Every frame, check whether the submarine is inside the current zone. If yes, add the push to velocity.

What You Should Avoid Mentioning in the Final Submission

Avoid saying:

GL_FOG
GL_LIGHTING
transparent sonar sphere
real lighting
real shadows
texture mapping

Unless your teacher explicitly allows those functions.

Use safer wording:

simulated fog
depth darkness
fake headlight beam
visual sonar pulse
color-based glow
primitive-based 3D environment

Recommended Revised Unique Feature

Your unique feature is still strong, but I would rewrite it like this:

Unique Feature

Hydrodynamic 3D Movement with Environmental Currents

Unlike simple 2D movement games, this project allows the submarine to move freely through a 3D underwater space. The submarine has velocity, momentum, and water resistance, so it continues drifting after thrust is released. Invisible ocean current zones push the submarine off course, forcing the player to carefully balance thrust, turning, elevation, and collision avoidance.

This creates a more realistic underwater navigation challenge while staying within the allowed OpenGL primitive and transformation functions.

Best Final Version of Your Project Claim

You can say this in your proposal:

Our project will use only the OpenGL/GLUT functions demonstrated in the provided starter file, along with depth testing. Advanced effects such as fog, lighting, glow, and sonar transparency will be simulated using primitive shapes, color changes, transformations, and game-state calculations.

That sounds honest, practical, and still impressive.